PROGRAMACIÓN ENTORNOS MULTIMEDIA · SEMANA 9 (28 DE OCTUBRE 2026)

El Proyecto Integrador: Del Espacio Visual al Mundo Interactivo

Mecánicas interactivas avanzadas: Triggers, Zonas Sonoras y Portales de Teletransporte.

Docente: Mg. Gabriel Guerrero Arellano



"La semana pasada construimos el escenario visual. Hoy le daremos vida, cerebro y reglas matemáticas de interacción."

La Gran Transición: Momento 2 vs Momento 3

Momento 2

Lo que hicimos

Modelar en Blender, exportar a .glb e importar en Three.js. Éramos espectadores en un escenario estático.

Momento 3

Lo que empieza hoy

Convertir el parque en un videojuego/experiencia interactiva. El código responde a la posición del usuario.

01

Modelo 3D Visual

(.glb)

02

Zonas Lógicas

Triggers

03

Eventos

Audio / Portales



¿Qué es una Zona de Activación (Trigger Area)?

Definición: un área invisible en el espacio 3D que no bloquea el paso, pero dispara una acción cuando la cámara o entidad entra en ella.

Dos tarjetas comparativas: los 2 métodos matemáticos para calcular triggers en Three.js.



Distancia Radial

Vector3.distanceTo

Ideal para fuentes, fogatas o áreas circulares.

`distancia = posicionCamara.distanceTo(centroZona)`

Si `distancia < radio`, se activa el evento.



Cajas Delimitadoras AABB

Box3

Cajas invisibles con coordenadas mínimas y máximas (min, max). Ideales para puertas, portales o cuartos rectangulares.

```
zonaBox.containsPoint(camera.position)
```

Organización de Archivos del Proyecto Integrador

```
Lab_Semana_9_Proyecto/
 ├── index.html          (Lienzo con CDN Three.js)
 ├── main.js             (Lógica, triggers y bucle principal)
 ├── style.css           (Interfaz UI / HUD en pantalla)
 ├── metaverso.glb       (El modelo optimizado de la Semana 8)
 └── sounds/             (Recursos de audio para triggers)
     ├── ambiente.mp3
     └── teleport.mp3
```

⚠ **Regla estricta: todos los nombres en minúsculas y sin espacios para evitar errores 404 en el servidor local**

Paso 0: Recursos y Descarga de Audios (.mp3)

Para nuestras zonas sonoras necesitamos efectos de sonido en formato .mp3 o .ogg.

Plataformas gratuitas recomendadas

1. Pixabay Sound Effects

pixabay.com/sound-effects

Buscar *"fountain water"* o *"portal magic"*. Se descarga directo en .mp3.

2. Mixkit

mixkit.co/free-sound-effects

Excelente banco de sonidos FX para videojuegos sin registro.

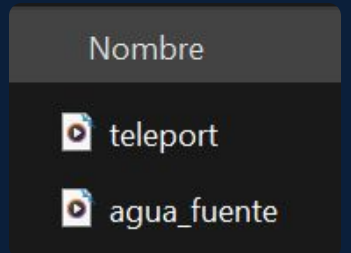
3. Freesound

freesound.org

Repositorio global de audio espacial.

Organización en VS Code

- Crea una carpeta llamada `sounds/` en la raíz de tu proyecto.
- Descarga los audios y guárdalos con nombres simples, en minúsculas y sin espacios:
  - `sounds/agua_fuente.mp3` (Efecto ambiental para la fuente)
  - `sounds/teleport.mp3` (Efecto opcional para el portal)



🧱 Paso 1: Lienzo HTML e Interfaz HUD (index.html) · Parte 1 de 2

📍 Ubicación: `index.html`

Preparamos el contenedor 3D, el Importmap de Three.js y una interfaz flotante en pantalla (HUD) para monitorear las zonas y coordenadas en tiempo real.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Proyecto Integrador - Semana 9</title>
    <style>
        body { margin: 0; overflow: hidden; font-family: Arial, sans-serif; }
        canvas { display: block; }
        /* Panel HUD de depuración sobre el canvas */
        #debug-ui {
            position: absolute;
            top: 20px;
            left: 20px;
            padding: 15px;
            background: rgba(0, 0, 0, 0.8);
            color: #00ffcc;
            border-radius: 8px;
            border: 1px solid #00ffcc;
            pointer-events: none;
            z-index: 10;
        }
        /* Banner para indicar el desbloqueo de audio */
        #audio-banner {
            position: absolute;
            bottom: 20px;
            left: 50%;
            transform: translateX(-50%);
            background: rgba(255, 204, 0, 0.9);
            color: #000;
            padding: 10px 20px;
            border-radius: 20px;
            font-weight: bold;
            font-size: 14px;
            pointer-events: none;
            z-index: 10;
        }
    </style></head>
```

🧱 Paso 1: Lienzo HTML e Interfaz HUD (index.html) · Parte 2 de 2

📍 Ubicación: `index.html`

Continuación: el cuerpo de la página con el HUD, el banner de audio, el Importmap y la carga de `main.js`.

```
<body>
    <div id="debug-ui">
        <p style="margin: 0 0 5px 0;"><strong>Zona Actual:</strong> <span id="zona-nombre">Explorando el Parque</span></p>
        <p style="margin: 0;"><strong>Posición:</strong> <span id="pos-info">X: 0 | Z: 0</span></p>
    </div>
    <div id="audio-banner">Haz clic en la pantalla para activar el audio posicional</div>
    <script type="importmap">
    {
        "imports": {
            "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
            "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
        }
    }
    </script>
    <script type="module" src="main.js"></script>
</body>
</html>
```

⚙️ Paso 2: Carga del Escenario Base (main.js)

📍 Ubicación: `main.js` (Parte superior)

Inicializamos la escena, cámara, renderizador, controles de órbita y cargamos el modelo `.glb` exportado de Blender.

```
import * as THREE from 'three';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
// 1. ESCENA, CÁMARA Y RENDERIZADOR
const scene = new THREE.Scene();
// CIELO AZUL VIBRANTE Y NIEBLA ATMOSFÉRICA DE PROFUNDIDAD
scene.background = new THREE.Color(0x38bdf8); // Azul cielo veraniego brillante
scene.fog = new THREE.FogExp2(0x38bdf8, 0.015); // Niebla suave que difumina el horizonte
const camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1, 1000);
camera.position.set(0, 5, 15); // Elevada y alejada para ver el parque
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);
// 2. CONTROLES Y LUCES
const controls = new OrbitControls(camera, renderer.domElement);
controls.target.set(0, 1, 0);
controls.update();
const luzAmbiental = new THREE.AmbientLight(0xffffff, 0.7);
scene.add(luzAmbiental);
const luzSol = new THREE.DirectionalLight(0xffffff, 1.2);
luzSol.position.set(10, 20, 10);
scene.add(luzSol);
// 3. CARGA DEL MODELO .GLB
const loader = new GLTFLoader();
loader.load('metaverso.glb', (gltf) => {
    scene.add(gltf.scene);
    console.log("¡Escenario del Parque cargado!");
});
```

🎧 Paso 3: Configurar el Audio Listener (main.js) Parte 1 de 2

📍 **Ubicación:** `main.js` (A continuación)

Acoplamos el receptor de sonido a la cámara y cargamos los audios de la fuente y del portal.

```
// 4. CONFIGURACIÓN DEL RECEPTOR DE AUDIO
const listener = new THREE.AudioListener();
camera.add(listener); // Acoplar los "oídos" a la cámara
const sonidoFuente = new THREE.PositionalAudio(listener);
const audioLoader = new THREE.AudioLoader();
let audioIniciado = false;
// Carga del archivo de audio descargado en el Paso 0
audioLoader.load('sounds/agua_fuente.mp3', function(buffer) {
    sonidoFuente.setBuffer(buffer);
    sonidoFuente.setRefDistance(3);   // Distancia donde se escucha al 100%
    sonidoFuente.setMaxDistance(12);  // Límite donde deja de escucharse
    sonidoFuente.setLoop(true);
    sonidoFuente.setVolume(0.8);
});


// ============================================================
// (SONIDO DEL PORTAL)
// ============================================================
const sonidoPortal = new THREE.Audio(listener);
audioLoader.load('sounds/teleport.mp3', (buffer) => {
    sonidoPortal.setBuffer(buffer);
    sonidoPortal.setVolume(0.8);
});
// ============================================================
```

🎧 Paso 3: Configurar el Audio Listener (main.js) Parte 2 de 2

📍 **Ubicación:** `main.js` (A continuación)

Gestionamos el desbloqueo de *Autoplay* del navegador al primer clic.

```
// Desbloquear el contexto de audio con un clic en la pantalla
window.addEventListener('click', () => {
    if (!audioIniciado && sonidoFuente.buffer) {
        sonidoFuente.play();
        audioIniciado = true;
        const banner = document.getElementById('audio-banner');
        if (banner) banner.style.display = 'none';
    }
});
// Desbloquear el contexto de audio con un clic en la pantalla
window.addEventListener('click', () => {
    if (!audioIniciado && sonidoFuente.buffer) {
        sonidoFuente.play();
        audioIniciado = true;
        const banner = document.getElementById('audio-banner');
        if (banner) banner.style.display = 'none';
    }
}
);
```

📍 Paso 4: Ubicación del Emisor 3D (main.js)

📍 **Ubicación:** `main.js` **(A continuación)**

Conectamos la fuente de audio posicional a un emisor 3D ubicado sobre la fuente de agua del parque.

> ⓘ ✅ **Criterio de verificación: Al orbitar la cámara cerca del centro, el sonido sube; al alejarla hacia los pilares o esquinas, disminuye de forma natural.**

```
// 5. ANCLAR EL SONIDO A LA COORDENADA DE LA FUENTE
const emisorFuente = new THREE.Object3D();
emisorFuente.position.set(0, 1, 0); // Altura de la copa de la fuente (X:0, Y:1, Z:0)
emisorFuente.add(sonidoFuente);
scene.add(emisorFuente);
```

🌀 Paso 5: Portal y Bounding Box (main.js)

📍 **Ubicación:** `main.js` (¡Debe ir ANTES de la función animate()!)

Creamos el objeto del portal, definimos su caja delimitadora invisible (`Box3`) y declaramos la variable de enfriamiento (*cooldown*).

```
// 6. CREACIÓN VISUAL DEL PORTAL (Anillo neón)
const portalGeo = new THREE.TorusGeometry(1.2, 0.15, 16, 50);
const portalMat = new THREE.MeshBasicMaterial({
    color: 0x00ffff,
    wireframe: true
});
const portal = new THREE.Mesh(portalGeo, portalMat);
portal.position.set(-7, 1.5, -7); // Esquina trasera izquierda
scene.add(portal);
// CAJA DELIMITADORA INVISIBLE (Trigger AABB)
const portalBox = new THREE.Box3();
portalBox.setFromCenterAndSize(
    new THREE.Vector3(-7, 1.5, -7),
    new THREE.Vector3(2, 3, 2)
);
// Variables de estado
let puedeTeletransportar = true;
const posFuente = new THREE.Vector3(0, 0, 0); // Centro de la fuente para el radio
```

🔁 Paso 6: Bucle animate() y Eventos (main.js) · Parte 1 de 2

📍 **Ubicación:** `main.js` **(Al final del archivo)**

Evaluamos en cada fotograma la distancia a la fuente y si la cámara cruzó el portal.

```
// 7. BUCLE PRINCIPAL DE ANIMACIÓN Y COMPROBACIÓN DE EVENTOS
function animate() {
    requestAnimationFrame(animate);
    // Animación del portal
    portal.rotation.z += 0.02;
    // Actualizar coordenadas en la interfaz HUD
    const posInfo = document.getElementById("pos-info");
    if (posInfo) {
        posInfo.innerText = `X: ${camera.position.x.toFixed(1)} | Z: ${camera.position.z.toFixed(1)}`;
    }
    // TRIGGER 1: Detección Radial de la Fuente
    const distancia = camera.position.distanceTo(posFuente);
    const zonaNombre = document.getElementById("zona-nombre");
    if (zonaNombre) {
        if (distancia < 5) {
            zonaNombre.innerText = "En el área de la fuente";
        } else {
            zonaNombre.innerText = "Paseando por el parque";
        }
    }
```

🔁 Paso 6: Bucle animate() y Eventos (main.js) · Parte 2 de 2

📍 Ubicación: `main.js` (Al final del archivo)

Continuación: detección del portal con la caja delimitadora, render y eventos de ventana.

```
    // TRIGGER 2: Teletransporte por Bounding Box
    if (portalBox.containsPoint(camera.position) && puedeTeletransportar) {
        puedeTeletransportar = false;


        // REPRODUCIR SONIDO DEL PORTAL
        if (sonidoPortal && sonidoPortal.buffer) {
            if (sonidoPortal.isPlaying) sonidoPortal.stop();
            sonidoPortal.play();
        }


        if (zonaNombre) {
            zonaNombre.innerText = "¡Portal Activado! Teletransportando...";
        }

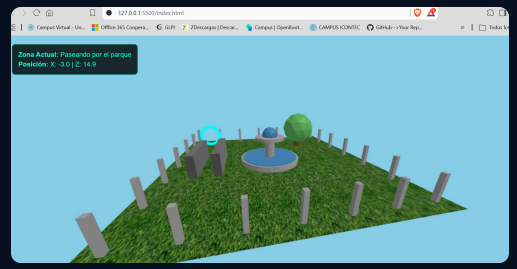

        // Teletransportar cámara y reorientar controles
        camera.position.set(7, 4, 7);
        controls.target.set(0, 1, 0);
        controls.update();


        // Tiempo de espera (cooldown) de 3 segundos
        setTimeout(() => {
            puedeTeletransportar = true;
        }, 3000);
    }
    renderer.render(scene, camera);
}
// Iniciar bucle de animación
animate();
// Ajuste Responsive al redimensionar ventana
window.addEventListener('resize', () => {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
});
```

🧪 Paso 7: Prueba en Vivo con Live Server

1
🌐 Abrir con Live Server

En VS Code, haz clic derecho sobre **index.html** y selecciona Open with Live Server.



2
👆 Activar el audio

Haz un clic sobre la pantalla para activar el sistema de audio posicional.

3
🔊 Verificación de Audio (Trigger Radial)

Acerca la cámara al centro del parque; el HUD debe marcar *"En el área de la fuente"* y el volumen del agua debe subir.

4
🌀 Verificación del Portal (Trigger Box3)

Mueve la cámara hacia el anillo neón en la esquina $(-7, 1.5, -7)$. Al tocarlo, el HUD cambiará, la cámara se moverá a la esquina opuesta $(7, 4, 7)$ y esperará 3 segundos antes de reactivarse.

Mejora Visual (Paso 2): Creación de la Fuente de Agua

▸ **Ubicación:** `Archivo main.js` · En el bloque global de configuración, AFUERA y ANTES de la función animate() (justo debajo de la creación del portal y los triggers)

```
// ========================================================
// MEJORA 2: CREACIÓN DEL SISTEMA DE PARTÍCULAS (FUERA DE ANIMATE)
// ========================================================
const numParticulas = 250;
const geometriaAgua = new THREE.BufferGeometry();
const posicionesAgua = new Float32Array(numParticulas * 3);
const velocidadesAgua = [];
for (let i = 0; i < numParticulas; i++) {
    // Posición inicial cerca de la copa de la fuente (Y: 1.8)
    posicionesAgua[i * 3] = (Math.random() - 0.5) * 0.3;
    posicionesAgua[i * 3 + 1] = 1.8 + Math.random() * 0.5;
    posicionesAgua[i * 3 + 2] = (Math.random() - 0.5) * 0.3;
    // Impulso inicial hacia arriba y dispersión horizontal
    velocidadesAgua.push({
        x: (Math.random() - 0.5) * 0.03,
        y: 0.04 + Math.random() * 0.04,
        z: (Math.random() - 0.5) * 0.03
    });
}
geometriaAgua.setAttribute('position', new THREE.BufferAttribute(posicionesAgua, 3));
const materialAgua = new THREE.PointsMaterial({
    color: 0x7dd3fc, // Azul celeste brillante
    size: 0.15,
    transparent: true,
    opacity: 0.85
});
const chorroFuente = new THREE.Points(geometriaAgua, materialAgua);
scene.add(chorroFuente);
```

Explicación

¿Por qué va AFUERA de animate()? Este bloque construye los 250 objetos geométricos de las gotas una sola vez en memoria. Si lo pusiéramos dentro del bucle animate(), crearíamos miles de objetos nuevos por segundo hasta congelar el navegador.

Mejora Visual (Paso 3): Animación del Agua y Gravedad

▸ Ubicación: `Archivo main.js` · DENTRO de la función animate() (en las primeras líneas del bucle, justo después de `portal.rotation.z += 0.02;`)

```
function animate() {
    requestAnimationFrame(animate);
    // Rotación del portal
    portal.rotation.z += 0.02;
    // ============================================================
    // MEJORA 3: ANIMACIÓN DEL AGUA Y CAÍDA DE GOTAS (DENTRO DE ANIMATE)
    // ============================================================
    const posiciones = chorroFuente.geometry.attributes.position.array;
```

```
for (let i = 0; i < numParticulas; i++) {
    // 1. Aplicar movimiento
    posiciones[i * 3] += velocidadesAgua[i].x;
    posiciones[i * 3 + 1] += velocidadesAgua[i].y;
    posiciones[i * 3 + 2] += velocidadesAgua[i].z;
    // 2. Gravedad (atrae las gotas hacia abajo)
    velocidadesAgua[i].y -= 0.0018;
    // 3. Reinicio: Si la gota cae al plato (Y < 1.0), vuelve a subir
    if (posiciones[i * 3 + 1] < 1.0) {
        posiciones[i * 3] = (Math.random() - 0.5) * 0.2;
        posiciones[i * 3 + 1] = 1.8;
        posiciones[i * 3 + 2] = (Math.random() - 0.5) * 0.2;
        velocidadesAgua[i].y = 0.04 + Math.random() * 0.04;
    }
}
// Notificar a Three.js que las posiciones cambiaron en este frame
chorroFuente.geometry.attributes.position.needsUpdate = true;
// ... (Sigue el código de HUD, Triggers y renderer.render) ...
```

Explicación · ¿Por qué va DENTRO de animate()? Se ejecuta 60 veces por segundo: aplica la gravedad y actualiza la posición de cada gota. needsUpdate = true ordena a la GPU volver a dibujar las partículas en su nueva posición.

Depuración de Errores Comunes (Checklist)

Problema en Pantalla	Causa Raíz	Solución Inmediata
No suena nada.	El navegador bloqueó el *Autoplay*.	Hacer clic en la pantalla para activar el `AudioContext`.
`Cannot access 'portalBox' before initialization`.	Se declaró `portalBox` al final del código, debajo de `animate()`.	Mover la creación de `portal` y `portalBox` arriba de la función `animate()`.
El portal teletransporta en bucle infinito.	Falta el control con la variable `puedeTeletransportar`.	Usar `puedeTeletransportar = false` y el `setTimeout` de 3 segundos.
La cámara salta y gira desorientada.	Se cambió `camera.position` pero no `controls.target`.	Llamar a `controls.target.set(0, 1, 0)` y luego `controls.update()`.

📋 Evidencia de la Sesión y Preparación para la Semana 10

✅ Entregable de la Clase

- Escenario corriendo en `localhost` con el HUD de depuración activo.
- Demostración en vivo de cambio de volumen por distancia en la fuente.
- Teletransporte funcional mediante el portal con tiempo de espera (*cooldown*).

🚀 Próxima Clase (Semana 10)

- Integraremos un personaje avatar en tercera persona con animaciones de caminata/reposo.
- Controles con teclado (WASD) y colisiones sólidas con los muros de nuestro parque.